



南開大學
Nankai University

数据结构实验报告三

矩阵

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络空间安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	2
1.4 代码	3
1.5 测试用例截图	10
1.6 复杂度分析	11
2 题目二	11
2.1 题目表述	11
2.2 代码 2 的测试用例	11
2.3 思路	11
2.4 代码	12
2.5 测试用例截图	15
2.6 复杂度分析	15
2.6.1 空间复杂度	15
2.6.2 时间复杂度	16
3 题目三	16
3.1 题目表述	16
3.2 代码 3 的测试用例	16
3.3 思路	16
3.4 代码	17
3.5 测试用例截图	24
3.6 复杂度分析	24

1 题目一

1.1 题目表述

扩充类 `lowerTriangularMatrix`, 增加矩阵转置方法, 返回值是下三角矩阵的转置矩阵, 是上三角矩阵, 是类 `upperTriangularMatrix` 的一个实例。确定时间复杂度。

1.2 代码 1 的测试用例

1. 输入: $L = [[5]]$; 输出: $U = [[5]]$, 类型为 `upperTriangularMatrix` (另外一个类, 不是原类)
2. 输入: $L = [[1, 0, 0], [4, 2, 0], [7, 5, 3]]$; 输出: $[[1, 4, 7], [0, 2, 5], [0, 0, 3]]$, 类型为 `upperTriangularMatrix`。
3. 输入: $L = [[0, 0], [0, 0]]$; 输出: $[[0, 0], [0, 0]]$, 类型为 `upperTriangularMatrix`。
4. 输入: $L = [[1, 9, 0], [4, 2, 0], [7, 5, 3]]$, 输出: 抛出异常或者 `assert` 失败。
5. 输入: $L = [[1, 0, 0], [4, 2, 0], [7, 5, 3]]$, 判断 $L.transpose().transpose() == L$ 。

1.3 思路

下三角矩阵只存 $n(n+1)/2$ 个 $i \geq j$ 的元素。使用一维 `vector<int> data` 存储, 索引公式为:

- 下三角: 第 i 行前 i 行共 $i*(i+1)/2$ 个, 加上 $j \rightarrow i*(i+1)/2 + j$
- 上三角: 第 j 列前 j 列共 $j*(j+1)/2$ 个, 加上 $i \rightarrow j*(j+1)/2 + i$

先定义两个核心类 `UpperTriangularMatrix` 和 `LowerTriangularMatrix`, 都使用一维数组压缩存储。再实现矩阵转置方法 `UpperTriangularMatrix::transpose()` 和 `LowerTriangularMatrix::transpose()`, 二者相互照应。

1. `UpperTriangularMatrix::transpose()`:

- 遍历所有 $i \leq j$ 的元素
- 将 $U[i][j]$ 的值赋给新 `LowerTriangularMatrix` 的 $L[j][i]$
- 返回一个下三角矩阵

1. `LowerTriangularMatrix::transpose()`:

- 遍历所有 $i \geq j$ 的元素
- 将 $L[i][j]$ 的值赋给新 `UpperTriangularMatrix` 的 $U[j][i]$
- 返回一个上三角矩阵

之后编写输入解析函数 `parseInput`, 把用户输入的字符串转换成二维 `vector<vector<int>>`, 去除所有空格和制表符, 检查是否以 `[` 开头并逐行解析。最后进入主函数 `main()`。

1.4 代码

```
1 #include <iostream>
2 #include <vector>
3 #include <sstream>
4 #include <stdexcept>
5 #include <cctype>
6 using namespace std;
7 class UpperTriangularMatrix;
8 class LowerTriangularMatrix {
9 private:
10     int n;
11     vector<int> data;
12 public:
13     LowerTriangularMatrix(int size) : n(size) {
14         data.resize(n * (n + 1) / 2, 0);
15     }
16     static LowerTriangularMatrix fromVector(const vector<vector<int>>& matrix) {
17         int size = matrix.size();
18         if (size == 0) throw invalid_argument("Matrix is empty");
19
20         for (const auto& row : matrix) {
21             if (row.size() != size) {
22                 throw invalid_argument("Matrix must be square");
23             }
24         }
25
26         for (int i = 0; i < size; i++) {
27             for (int j = i + 1; j < size; j++) {
28                 if (matrix[i][j] != 0) {
29                     throw invalid_argument("Matrix is not lower triangular");
30                 }
31             }
32         }
33
34         LowerTriangularMatrix result(size);
35         for (int i = 0; i < size; i++) {
36             for (int j = 0; j <= i; j++) {
37                 result.set(i, j, matrix[i][j]);
38             }
39         }
40         return result;
```

```
41     }
42
43     int get(int i, int j) const {
44         if (i < 0 || i >= n || j < 0 || j >= n) {
45             throw out_of_range("Index out of bounds");
46         }
47         if (i >= j) {
48             return data[i * (i + 1) / 2 + j];
49         }
50         else {
51             return 0;
52         }
53     }
54
55     void set(int i, int j, int value) {
56         if (i < 0 || i >= n || j < 0 || j >= n) {
57             throw out_of_range("Index out of bounds");
58         }
59         if (i >= j) {
60             data[i * (i + 1) / 2 + j] = value;
61         }
62         else {
63             throw invalid_argument("Can only set elements in lower triangular part");
64         }
65     }
66     UpperTriangularMatrix transpose() const;
67     vector<vector<int>> toVector() const {
68         vector<vector<int>> result(n, vector<int>(n, 0));
69         for (int i = 0; i < n; i++) {
70             for (int j = 0; j <= i; j++) {
71                 result[i][j] = get(i, j);
72             }
73         }
74         return result;
75     }
76
77     void display() const {
78         auto vec = toVector();
79         cout << "[";
80         for (int i = 0; i < n; i++) {
81             cout << "[";
82             for (int j = 0; j < n; j++) {
```

```
83         cout << vec[i][j];
84         if (j < n - 1) cout << ", ";
85     }
86     cout << "]\n";
87     if (i < n - 1) cout << ", ";
88 }
89 cout << "]\n" << endl;
90 }
91
92 int size() const { return n; }
93 bool operator==(const LowerTriangularMatrix& other) const {
94     if (n != other.n) return false;
95     for (int i = 0; i < n; i++) {
96         for (int j = 0; j <= i; j++) {
97             if (get(i, j) != other.get(i, j)) {
98                 return false;
99             }
100         }
101     }
102     return true;
103 }
104 };
105
106 class UpperTriangularMatrix {
107 private:
108     int n;
109     vector<int> data;
110 public:
111     UpperTriangularMatrix(int size) : n(size) {
112         data.resize(n * (n + 1) / 2, 0);
113     }
114
115     static UpperTriangularMatrix fromVector(const vector<vector<int>>& matrix) {
116         int size = matrix.size();
117         if (size == 0) throw invalid_argument("Matrix is empty");
118
119         for (const auto& row : matrix) {
120             if (row.size() != size) {
121                 throw invalid_argument("Matrix must be square");
122             }
123         }
124     }
```

```
125     for (int i = 0; i < size; i++) {
126         for (int j = 0; j < i; j++) {
127             if (matrix[i][j] != 0) {
128                 throw invalid_argument("Matrix is not upper triangular");
129             }
130         }
131     }
132
133     UpperTriangularMatrix result(size);
134     for (int i = 0; i < size; i++) {
135         for (int j = i; j < size; j++) {
136             result.set(i, j, matrix[i][j]);
137         }
138     }
139     return result;
140 }
141
142 int get(int i, int j) const {
143     if (i < 0 || i >= n || j < 0 || j >= n) {
144         throw out_of_range("Index out of bounds");
145     }
146     if (i <= j) {
147         return data[j * (j + 1) / 2 + i];
148     }
149     else {
150         return 0;
151     }
152 }
153
154 void set(int i, int j, int value) {
155     if (i < 0 || i >= n || j < 0 || j >= n) {
156         throw out_of_range("Index out of bounds");
157     }
158     if (i <= j) {
159         data[j * (j + 1) / 2 + i] = value;
160     }
161     else {
162         throw invalid_argument("Can only set elements in upper triangular part");
163     }
164 }
165
166 LowerTriangularMatrix transpose() const {
```

```
167     LowerTriangularMatrix result(n);
168     for (int i = 0; i < n; i++) {
169         for (int j = i; j < n; j++) {
170             result.set(j, i, get(i, j));
171         }
172     }
173     return result;
174 }
175
176 vector<vector<int>> toVector() const {
177     vector<vector<int>> result(n, vector<int>(n, 0));
178     for (int i = 0; i < n; i++) {
179         for (int j = i; j < n; j++) {
180             result[i][j] = get(i, j);
181         }
182     }
183     return result;
184 }
185
186 void display() const {
187     auto vec = toVector();
188     cout << "[";
189     for (int i = 0; i < n; i++) {
190         cout << "[";
191         for (int j = 0; j < n; j++) {
192             cout << vec[i][j];
193             if (j < n - 1) cout << ", ";
194         }
195         cout << "];";
196         if (i < n - 1) cout << ", ";
197     }
198     cout << "]" << endl;
199 }
200
201 int size() const { return n; }
202 bool operator==(const UpperTriangularMatrix& other) const {
203     if (n != other.n) return false;
204     for (int i = 0; i < n; i++) {
205         for (int j = i; j < n; j++) {
206             if (get(i, j) != other.get(i, j)) {
207                 return false;
208             }
209         }
210     }
```

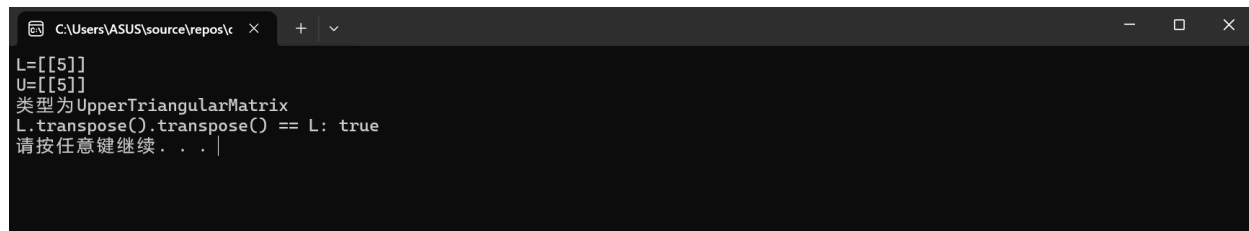


```
209     }
210 }
211 return true;
212 }
213 };
214
215 UpperTriangularMatrix LowerTriangularMatrix::transpose() const {
216     UpperTriangularMatrix result(n);
217     for (int i = 0; i < n; i++) {
218         for (int j = 0; j <= i; j++) {
219             result.set(j, i, get(i, j));
220         }
221     }
222     return result;
223 }
224
225 vector<vector<int>> parseInput(const string& input) {
226     vector<vector<int>> matrix;
227     string cleanStr;
228     for (char c : input) {
229         if (!isspace(c)) cleanStr += c;
230     }
231     if (cleanStr.empty() || cleanStr[0] != '[') {
232         throw invalid_argument("Invalid format: must start with '['");
233     }
234     size_t pos = 1;
235     while (pos < cleanStr.length()) {
236         if (cleanStr[pos] == '[') {
237             pos++;
238             vector<int> row;
239             string numStr;
240
241             while (pos < cleanStr.length() && cleanStr[pos] != ']') {
242                 if (isdigit(cleanStr[pos]) || cleanStr[pos] == '-') {
243                     numStr += cleanStr[pos];
244                 }
245                 else if (cleanStr[pos] == ',' || cleanStr[pos] == ' ') {
246                     if (!numStr.empty()) {
247                         row.push_back(stoi(numStr));
248                         numStr.clear();
249                     }
250                 }
251             }
```

```
251         pos++;
252     }
253
254     if (!numStr.empty()) {
255         row.push_back(stoi(numStr));
256     }
257
258     if (pos < cleanStr.length() && cleanStr[pos] == ']') {
259         pos++;
260     }
261
262     if (!row.empty()) {
263         matrix.push_back(row);
264     }
265     while (pos < cleanStr.length() && (cleanStr[pos] == ',' || cleanStr[pos]
        == ' ')) {
266         pos++;
267     }
268 }
269 else {
270     break;
271 }
272 }
273 return matrix;
274 }
275
276 int main() {
277     string input;
278     cout << "L=";
279     getline(cin, input);
280     try {
281         vector<vector<int>> matrixData = parseInput(input);
282         LowerTriangularMatrix lower = LowerTriangularMatrix::fromVector(matrixData);
283         UpperTriangularMatrix upper = lower.transpose();
284         cout << "U=";
285         upper.display();
286         cout << "类型为UpperTriangularMatrix" << endl;
287         LowerTriangularMatrix roundtrip = upper.transpose(); // U 转置回 L
288         cout << "L.transpose().transpose() == L: " << (lower == roundtrip ? "true" :
            "false") << endl;
289     }
290     catch (const exception& e) {
```

```
291     cout << "错误: " << e.what() << endl;  
292 }  
293 system("pause");  
294 return 0;  
295 }
```

1.5 测试用例截图



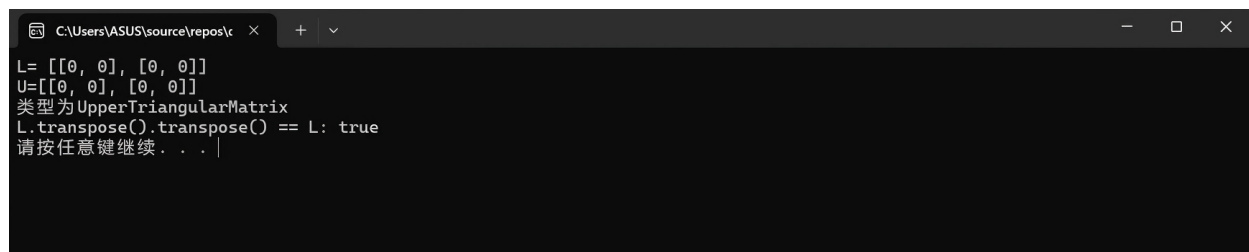
```
C:\Users\ASUS\source\repos\c x + v  
L=[[5]]  
U=[[5]]  
类型为UpperTriangularMatrix  
L.transpose().transpose() == L: true  
请按任意键继续. . . |
```

图 1.1: 题目一测试用例输出结果 1



```
C:\Users\ASUS\source\repos\c x + v  
L=[[1, 0, 0], [4, 2, 0], [7, 5, 3]]  
U=[[1, 4, 7], [0, 2, 5], [0, 0, 3]]  
类型为UpperTriangularMatrix  
L.transpose().transpose() == L: true  
请按任意键继续. . . |
```

图 1.2: 题目一测试用例输出结果 2



```
C:\Users\ASUS\source\repos\c x + v  
L= [[0, 0], [0, 0]]  
U=[[0, 0], [0, 0]]  
类型为UpperTriangularMatrix  
L.transpose().transpose() == L: true  
请按任意键继续. . . |
```

图 1.3: 题目一测试用例输出结果 3



```
C:\Users\ASUS\source\repos\c x + v  
L=[[1, 9, 0], [4, 2, 0], [7, 5, 3]]  
错误: Matrix is not lower triangular  
请按任意键继续. . . |
```

图 1.4: 题目一测试用例输出结果 4

1.6 复杂度分析

总时间复杂度为 $O(n^2)$:

- `parseInput(input)` 部分时间复杂度 $O(n^2)$

设输入矩阵为 $n \times n$, 则字符串长度大致为 $O(n^2)$ 。需要遍历整个字符串一次, 每个数字转换为整数是常数时间。

- `LowerTriangularMatrix::fromVector(matrix)` 构造下三角矩阵时间复杂度 $O(n^2)$

检查方阵部分, 设有 n 行, 每行检查长度 $O(n)$; 上三角部分有 $n(n-1)/2$ 个元素, 需要遍历所有 $i=0$ 到 $n-1$, $j=i+1$ 到 $n-1$, 总共约 $O(n^2)$ 次检查; 填充数据时设下三角部分有 $n(n+1)/2$ 个元素, 每次 `set(i,j)` 是 $O(1)$ (直接计算索引), 总共 $O(n^2)$ 次赋值。

`LowerTriangularMatrix::transpose()` 转置为上三角矩阵同理。

- `UpperTriangularMatrix::transpose()` 转置回下三角矩阵时间复杂度 $O(n^2)$

元素个数: $n(n+1)/2$ $O(n^2)$, 每次 `set` 是 $O(1)$, 总共 $O(n^2)$ 次操作。

2 题目二

2.1 题目表述

编写一个方法, 把两个存储在一维数组的稀疏矩阵相乘。假定两个矩阵和结果矩阵都是按行主次序存储。

2.2 代码 2 的测试用例

1. 输入: $A = (1, 1, \text{triples}=[(0,0,5)])$ $B = (1, 1, \text{triples}=[(0,0,7)])$; 输出: $C = (1, 1, \text{triples}=[(0,0,35)])$
2. 输入: $A = (2, 3, \text{triples}=[(0,0,1), (1,2,2)])$ $B = (3, 2, \text{triples}=[(0,1,3), (2,0,4)])$; 输出: $C = (2, 2, \text{triples}=[(0,1,3), (1,0,8)])$
3. 输入: $A = (3, 3, \text{triples}=[(0,2,7), (1,1,5)])$ $B = (3, 3, \text{triples}=[(0,0,1), (1,1,1), (2,2,1)])$; 输出: $C = (3, 3, \text{triples}=[(0,2,7), (1,1,5)])$
4. 输入: $A = (2, 2, \text{triples}=[(0,1,5)])$ $B = (2, 2, \text{triples}=[(0,0,9)])$; 输出: $C = (2, 2, \text{triples}=[])$
5. 输入: $A = (2, 3, \text{triples}=[(0,0,1)])$ $B = (4, 2, \text{triples}=[(0,0,1)])$ 输出: 错误/异常 (维度不匹配: $A.\text{cols}=3$ $B.\text{rows}=4$)

2.3 思路

1. 先读取输入并解析字符串. 使用 `getline(cin, ...)` 读取整行, 再用 `istringstream` 将字符串转为可解析的流。手动跳过空格、读取括号、逗号等符号, 提取 `rows`、`cols` 和 `triples=[...]` 中的 `(r,c,val)` 列表。
2. 再检查矩阵维度是否匹配。直接比较 `A.cols` 和 `B.rows`。
3. 执行稀疏矩阵乘法。双重循环遍历非零元素, 使用 `unordered_map<int, unordered_map<int, double>> resultMap`。收集所有 `resultMap[i][j] != 0` 的项。构造 `vector<Triple>`, 并按行主序排序。

2.4 代码

```
1  #include <iostream>
2  #include <vector>
3  #include <unordered_map>
4  #include <algorithm>
5  #include <string>
6  #include <sstream>
7  #include <cctype>
8
9  using namespace std;
10
11 struct Triple {
12     int row, col;
13     double val;
14     Triple(int r, int c, double v) : row(r), col(c), val(v) {}
15 };
16
17 struct SparseMatrix {
18     int rows, cols;
19     vector<Triple> triples;
20     SparseMatrix(int r, int c) : rows(r), cols(c) {}
21     SparseMatrix(int r, int c, const vector<Triple>& t) : rows(r), cols(c),
22         triples(t) {}
23 };
24
25 SparseMatrix multiplySparseMatrices(const SparseMatrix& A, const SparseMatrix& B) {
26     if (A.cols != B.rows) throw invalid_argument("Matrix dimension mismatch");
27
28     unordered_map<int, unordered_map<int, double>> resultMap;
29     for (const auto& a : A.triples)
30         for (const auto& b : B.triples)
31             if (a.col == b.row)
32                 resultMap[a.row][b.col] += a.val * b.val;
33
34     vector<Triple> resultTriples;
35     for (const auto& ri : resultMap)
36         for (const auto& rj : ri.second)
37             if (rj.second != 0)
38                 resultTriples.emplace_back(ri.first, rj.first, rj.second);
39
40     sort(resultTriples.begin(), resultTriples.end(), [](const Triple& x, const
```

```
        Triple& y) {
40     return x.row == y.row ? x.col < y.col : x.row < y.row;
41     });
42
43     return SparseMatrix(A.rows, B.cols, resultTriples);
44 }
45 void skipSpaces(istream& is) {
46     while (is.peek() != EOF && isspace(is.peek())) is.get();
47 }
48 int parseInt(istream& is) {
49     int n; is >> n;
50     return n;
51 }
52 double parseDouble(istream& is) {
53     double d; is >> d;
54     return d;
55 }
56 Triple parseTriple(istream& is) {
57     is.get();
58     int r = parseInt(is);
59     is.get();
60     skipSpaces(is);
61     int c = parseInt(is);
62     is.get();
63     skipSpaces(is);
64     double v = parseDouble(is);
65     is.get();
66     return Triple(r, c, v);
67 }
68
69 vector<Triple> parseTriplesList(istream& is) {
70     string tag;
71     getline(is, tag, '=');
72     if (tag.find("triples") == string::npos) throw runtime_error("Invalid format:
        expected triples=");
73     is.get();
74     vector<Triple> triples;
75     char c = is.peek();
76     if (c == ']') {
77         is.get();
78         return triples;
79     }
```

```
80     while (true) {
81         triples.push_back(parseTriple(is));
82         c = is.peek();
83         if (c == ')') is.get();
84         if (c == ']') break;
85         is.get();
86         skipSpaces(is);
87     }
88     is.get();
89     return triples;
90 }
91 SparseMatrix parseMatrix() {
92     string input;
93     getline(cin, input);
94     istringstream is(input);
95
96     string name;
97     getline(is, name, '=');
98     skipSpaces(is);
99
100    is.get(); // '('
101    int rows = parseInt(is);
102    is.get(); // ','
103    skipSpaces(is);
104    int cols = parseInt(is);
105    is.get(); // ','
106    skipSpaces(is);
107
108    auto triples = parseTriplesList(is);
109    is.get(); // ')'
110
111    return SparseMatrix(rows, cols, triples);
112 }
113 void printMatrix(const SparseMatrix& mat) {
114     cout << "C = (" << mat.rows << ", " << mat.cols << ", triples=[";
115     for (size_t i = 0; i < mat.triples.size(); ++i) {
116         if (i > 0) cout << ", ";
117         const auto& t = mat.triples[i];
118         cout << "(" << t.row << ", " << t.col << ", " << t.val << ")";
119     }
120     cout << "]" << endl;
121 }
```

```
122 int main() {  
123     try {  
124         auto A = parseMatrix();  
125         auto B = parseMatrix();  
126         auto C = multiplySparseMatrices(A, B);  
127         printMatrix(C);  
128     }  
129     catch (const exception& e) {  
130         cout << "Error: " << e.what() << endl;  
131     }  
132     system("pause");  
133     return 0;  
134 }
```

2.5 测试用例截图

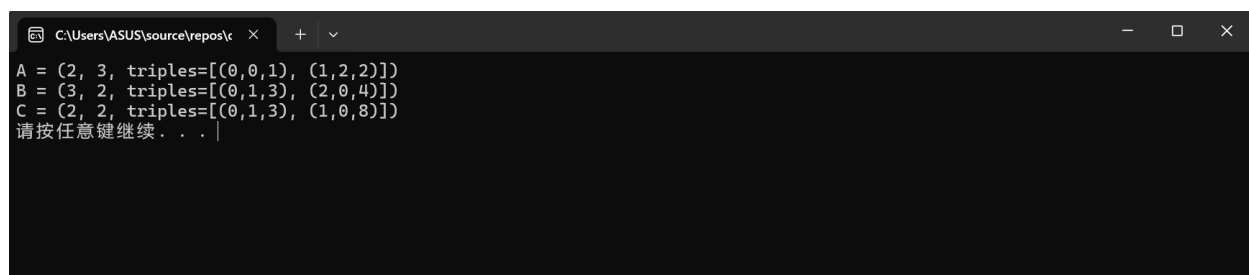


图 2.5: 题目二测试用例输出结果 1



图 2.6: 题目二测试用例输出结果 2

2.6 复杂度分析

2.6.1 空间复杂度

空间复杂度为 $O(\text{nnz}(A) + \text{nnz}(B) + \text{nnz}(C))$ 其中 $\text{nnz}(A)$ 、 $\text{nnz}(B)$ 和 $\text{nnz}(C)$ 分别表示矩阵 A 、 B 和结果矩阵 C 的非零元素个数。

2.6.2 时间复杂度

时间复杂度主要由稀疏矩阵乘法的核心计算决定，为 $O(\text{nnz}(A) \times \text{nnz}(B))$ 因为算法通过双重循环遍历 A 和 B 的所有非零元素，仅在行-列索引匹配时进行乘积累加。此外，最后对结果矩阵的非零元素按行主序排序，时间复杂度为 $O(\text{nnz}(C) \log \text{nnz}(C))$ 。因此，总时间复杂度为 $O(\text{nnz}(A) \times \text{nnz}(B) + \text{nnz}(C) \log \text{nnz}(C))$ 。

3 题目三

3.1 题目表述

给类 `linkedMatrix` 增加下列操作:

1. 已知一个元素的行、列和数值, 存储这个元素。
2. 已知一个元素的行和列, 从矩阵中取出这个元素。
3. 两个稀疏矩阵相加。
4. 两个稀疏矩阵相减。
5. 两个稀疏矩阵相乘。

细化操作符 » 的代码, 如练习 42 所要求的那样。并测试代码。

3.2 代码 3 的测试用例

空表 A 大小为 $3, 3$

1. 插入 $(0,2,7)$ 、 $(1,1,5)$ 、 $(0,1,3)$ 后输出
2. 取出 $(0,1)$ 并输出
3. 设 B 表为 $(3,3)$, $\text{triples} = [(0,2,7), (1,1,5)]$, 与 A 进行加法并输出
4. 做 $A-B$ 减法并输出
5. 做 A 和 B 的矩阵乘法并输出

3.3 思路

1. 定义稀疏矩阵的数据结构: 使用 ** 十字链表 (Orthogonal List) ** 来存储稀疏矩阵。

每个非零元素是一个节点 `Node`, 包含:

- `row, col`: 行列索引
- `val`: 值
- `right`: 指向同行下一个非零元
- `down`: 指向下一行同列非零元

维护两个数组:

- row_heads[]: 每行的头指针
 - col_heads[]: 每列的头指针
2. 插入元素调用 set 函数。set(r, c, v) 会查找第 r 行和第 c 列是否已有该位置的节点: 若无, 则创建新节点, 并将其插入到对应行链表和列链表中; 若有, 则更新值; 若值为 0, 则删除节点。
 3. 计算 $A + B$ 遍历 A 的每个非零元, 在 B 中查找对应位置的值, 相加后设入结果 C。再遍历 B 中 A 没有的非零元, 直接加入 C。
减法类似
 4. 计算 $A * B$ 预处理 B: 按列组织非零元 (otherCols[j] 存储第 j 列的所有 (row, val)), 对 A 的每一行, 提取非零元并构成哈希表 thisRow, 再对结果的每一列 j, 遍历 B 的第 j 列中的每个 (k, val), 累加 thisRow[k] * val。

3.4 代码

```
1  #include <iostream>
2  #include <vector>
3  #include <unordered_map>
4  #include <utility>
5  #include <sstream>
6  using namespace std;
7  struct Node {
8      int row, col, val;
9      Node* right;
10     Node* down;
11     Node(int r = 0, int c = 0, int v = 0) : row(r), col(c), val(v), right(nullptr),
        down(nullptr) {}
12 };
13
14 class LinkedMatrix {
15 private:
16     int rows, cols;
17     vector<Node*> row_heads;
18     vector<Node*> col_heads;
19
20     void insertInRow(Node* head, Node* node) {
21         Node* curr = head;
22         while (curr->right && curr->right->col < node->col)
23             curr = curr->right;
24         node->right = curr->right;
25         curr->right = node;
26     }
```

```
27
28 void insertInCol(Node* head, Node* node) {
29     Node* curr = head;
30     while (curr->down && curr->down->row < node->row)
31         curr = curr->down;
32     node->down = curr->down;
33     curr->down = node;
34 }
35
36 void addNode(int r, int c, int v) {
37     if (v == 0) return;
38     Node* node = new Node(r, c, v);
39     insertInRow(row_heads[r], node);
40     insertInCol(col_heads[c], node);
41 }
42
43 public:
44     LinkedMatrix(int r, int c) : rows(r), cols(c) {
45         row_heads.resize(rows);
46         col_heads.resize(cols);
47         for (int i = 0; i < rows; ++i)
48             row_heads[i] = new Node();
49         for (int j = 0; j < cols; ++j)
50             col_heads[j] = new Node();
51     }
52
53     LinkedMatrix(const LinkedMatrix& other) : rows(other.rows), cols(other.cols) {
54         row_heads.resize(rows);
55         col_heads.resize(cols);
56         for (int i = 0; i < rows; ++i)
57             row_heads[i] = new Node();
58         for (int j = 0; j < cols; ++j)
59             col_heads[j] = new Node();
60         for (int i = 0; i < other.rows; ++i) {
61             Node* curr = other.row_heads[i]->right;
62             while (curr) {
63                 set(curr->row, curr->col, curr->val);
64                 curr = curr->right;
65             }
66         }
67     }
68 }
```

```
69 LinkedMatrix& operator=(const LinkedMatrix& other) {
70     if (this == &other) return *this;
71     for (int i = 0; i < rows; ++i) {
72         Node* curr = row_heads[i]->right;
73         while (curr) {
74             Node* temp = curr;
75             curr = curr->right;
76             delete temp;
77         }
78     }
79     for (int i = 0; i < rows; ++i) delete row_heads[i];
80     for (int j = 0; j < cols; ++j) delete col_heads[j];
81
82     rows = other.rows;
83     cols = other.cols;
84     row_heads.resize(rows);
85     col_heads.resize(cols);
86     for (int i = 0; i < rows; ++i)
87         row_heads[i] = new Node();
88     for (int j = 0; j < cols; ++j)
89         col_heads[j] = new Node();
90     for (int i = 0; i < other.rows; ++i) {
91         Node* curr = other.row_heads[i]->right;
92         while (curr) {
93             set(curr->row, curr->col, curr->val);
94             curr = curr->right;
95         }
96     }
97     return *this;
98 }
99
100 ~LinkedMatrix() {
101     for (int i = 0; i < rows; ++i) {
102         Node* curr = row_heads[i]->right;
103         while (curr) {
104             Node* temp = curr;
105             curr = curr->right;
106             delete temp;
107         }
108         delete row_heads[i];
109     }
110     for (int j = 0; j < cols; ++j)
```

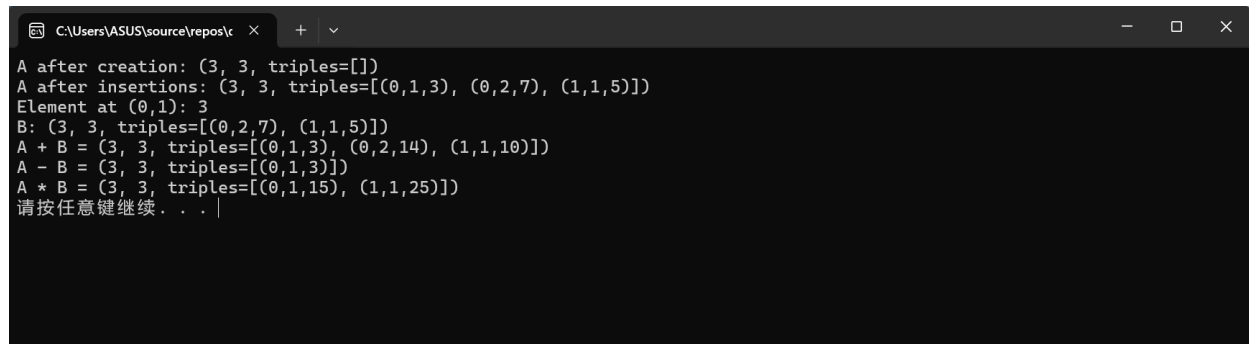
```
111         delete col_heads[j];
112     }
113
114     void set(int r, int c, int val) {
115         if (r < 0 || r >= rows || c < 0 || c >= cols) return;
116         Node* curr = row_heads[r]->right;
117         while (curr && curr->col < c)
118             curr = curr->right;
119         if (curr && curr->col == c) {
120             if (val == 0) {
121                 Node* prev_row = row_heads[r];
122                 while (prev_row->right != curr) prev_row = prev_row->right;
123                 prev_row->right = curr->right;
124                 Node* prev_col = col_heads[c];
125                 while (prev_col->down != curr) prev_col = prev_col->down;
126                 prev_col->down = curr->down;
127                 delete curr;
128             }
129             else {
130                 curr->val = val;
131             }
132         }
133         else {
134             if (val != 0) {
135                 addNode(r, c, val);
136             }
137         }
138     }
139     int get(int r, int c) const {
140         if (r < 0 || r >= rows || c < 0 || c >= cols) return 0;
141         Node* curr = row_heads[r]->right;
142         while (curr && curr->col < c)
143             curr = curr->right;
144         if (curr && curr->col == c) return curr->val;
145         return 0;
146     }
147     LinkedMatrix add(const LinkedMatrix& other) const {
148         if (rows != other.rows || cols != other.cols) {
149             cout << "Addition: Dimension mismatch.\n";
150             return *this;
151         }
152         LinkedMatrix result(rows, cols);
```

```
153     for (int i = 0; i < rows; ++i) {
154         Node* curr = row_heads[i]->right;
155         while (curr) {
156             int val = curr->val + other.get(curr->row, curr->col);
157             if (val != 0) result.set(curr->row, curr->col, val);
158             curr = curr->right;
159         }
160     }
161     for (int i = 0; i < other.rows; ++i) {
162         Node* curr = other.row_heads[i]->right;
163         while (curr) {
164             if (get(curr->row, curr->col) == 0) {
165                 result.set(curr->row, curr->col, curr->val);
166             }
167             curr = curr->right;
168         }
169     }
170     return result;
171 }
172 LinkedMatrix subtract(const LinkedMatrix& other) const {
173     if (rows != other.rows || cols != other.cols) {
174         cout << "Subtraction: Dimension mismatch.\n";
175         return *this;
176     }
177     LinkedMatrix result(rows, cols);
178     for (int i = 0; i < rows; ++i) {
179         Node* curr = row_heads[i]->right;
180         while (curr) {
181             int val = curr->val - other.get(curr->row, curr->col);
182             if (val != 0) result.set(curr->row, curr->col, val);
183             curr = curr->right;
184         }
185     }
186     for (int i = 0; i < other.rows; ++i) {
187         Node* curr = other.row_heads[i]->right;
188         while (curr) {
189             if (get(curr->row, curr->col) == 0) {
190                 result.set(curr->row, curr->col, -curr->val);
191             }
192             curr = curr->right;
193         }
194     }
```

```
195     return result;
196 }
197 LinkedMatrix multiply(const LinkedMatrix& other) const {
198     if (cols != other.rows) {
199         cout << "Multiplication: Incompatible dimensions.\n";
200         return LinkedMatrix(0, 0);
201     }
202     int newRows = rows;
203     int newCols = other.cols;
204     LinkedMatrix result(newRows, newCols);
205     vector<vector<pair<int, int>>> otherCols(newCols);
206     for (int k = 0; k < other.rows; ++k) {
207         Node* curr = other.row_heads[k]->right;
208         while (curr) {
209             otherCols[curr->col].push_back(make_pair(k, curr->val));
210             curr = curr->right;
211         }
212     }
213     for (int i = 0; i < rows; ++i) {
214         unordered_map<int, int> thisRow;
215         Node* curr = row_heads[i]->right;
216         while (curr) {
217             thisRow[curr->col] = curr->val;
218             curr = curr->right;
219         }
220         if (thisRow.empty()) continue;
221         for (int j = 0; j < newCols; ++j) {
222             int sum = 0;
223             for (const auto& p : otherCols[j]) {
224                 int k = p.first;
225                 int val = p.second;
226                 if (thisRow.find(k) != thisRow.end()) {
227                     sum += thisRow[k] * val;
228                 }
229             }
230             if (sum != 0) {
231                 result.set(i, j, sum);
232             }
233         }
234     }
235     return result;
236 }
```

```
237     string toString() const {
238         stringstream ss;
239         ss << "(" << rows << ", " << cols << ", triples=[";
240         bool first = true;
241         for (int i = 0; i < rows; ++i) {
242             Node* curr = row_heads[i]->right;
243             while (curr) {
244                 if (!first) ss << ", ";
245                 ss << "(" << curr->row << ", " << curr->col << ", " << curr->val << ")";
246                 first = false;
247                 curr = curr->right;
248             }
249         }
250         ss << "]";
251         return ss.str();
252     }
253 };
254 int main() {
255     LinkedMatrix A(3, 3);
256     cout << "A after creation: " << A.toString() << endl;
257     A.set(0, 2, 7);
258     A.set(1, 1, 5);
259     A.set(0, 1, 3);
260     cout << "A after insertions: " << A.toString() << endl;
261     int val = A.get(0, 1);
262     cout << "Element at (0,1): " << val << endl;
263     LinkedMatrix B(3, 3);
264     B.set(0, 2, 7);
265     B.set(1, 1, 5);
266     cout << "B: " << B.toString() << endl;
267     LinkedMatrix C = A.add(B);
268     cout << "A + B = " << C.toString() << endl;
269     LinkedMatrix D = A.subtract(B);
270     cout << "A - B = " << D.toString() << endl;
271     LinkedMatrix E = A.multiply(B);
272     cout << "A * B = " << E.toString() << endl;
273     system("pause");
274     return 0;
275 }
```


3.5 测试用例截图



```
C:\Users\ASUS\source\repos\c x + v
A after creation: (3, 3, triples=[])
A after insertions: (3, 3, triples=[(0,1,3), (0,2,7), (1,1,5)])
Element at (0,1): 3
B: (3, 3, triples=[(0,2,7), (1,1,5)])
A + B = (3, 3, triples=[(0,1,3), (0,2,14), (1,1,10)])
A - B = (3, 3, triples=[(0,1,3)])
A * B = (3, 3, triples=[(0,1,15), (1,1,25)])
请按任意键继续. . .
```

图 3.7: 题目三测试用例输出结果

3.6 复杂度分析

总空间复杂度 $O(\text{nnz} + \text{rows} + \text{cols})$

- `row_heads[]`: $O(\text{rows})$
- `col_heads[]`: $O(\text{cols})$
- 每个非零元一个节点: $O(\text{nnz})$